

WIP

November 18, 2023

[Click here for an up to date version](#)

Contents

1	Introduction	3
2	Basics	3
2.1	Vectors	3
2.2	Game updates	3
2.2.1	Simulating physics	4
2.2.2	Player tick	5
3	Jumping	6
4	Crouching	7
4.1	Crouch-tap jump	7
4.2	Headbug	8
5	Grounding	8
5.1	Rampbug	9
5.2	Speedshot	9
5.2.1	Velocity dependency	9
5.3	Bounce	9
5.3.1	Jumpbug and Glidestep	9
5.3.2	Viability	10
5.3.3	Chaining bounces	10
6	Projectiles and Knockback	11
6.1	Market Gardener bunnyhop	14
7	Acceleration	14
7.1	Strafing	15
7.1.1	Wall-hugging	15
8	Movement	15
8.1	Edgebug	16
9	Collision	17
9.1	Traces	18
9.2	Wallbug/Rampbug	20
9.2.1	Coordinate wallbug	21
9.3	Pixelwalk	22
9.4	Flush brushes	23
9.4.1	Pixelsurf	25
9.4.2	Texturebug	25

Name	Symbol	Value
Tick	T	-
Tick interval	δ	0.015
Ticks per second	δ^{-1}	66.6
Surface extension	ε	0.03125
Grounding height	h_{ground}	2
Max velocity	v_{max}	3500
Leave velocity	v_{leave}	250
Gravitational acceleration	g	800
Max velocity tick	T_{max}	-
Jump velocity	v_{jump}	289
Bunnyhop cap		120%
Walk speed	v_{walk}	Soldier 240
Coefficient of friction	μ	4
Stop speed	v_{stop}	100
Duck speed multiplier		1/3
Bounding box width	W	48
Bounding box height	H	Stand 82
		Duck 62
		Duck (old) 55
View height	h	Stand 68
		Duck 45
Aim extent	d_{aim}	2000
Rocket spawn offset	$\mathbf{s}$	$(s_f \ s_r \ s_u)$
	s_f	23.5
	s_r	Original 0
		Mangler 8
		Stock 12
	s_u	Stand -3
		Duck 8

Table 1

1 Introduction

Jumping in TF2 makes use of a lot of different movement-based mechanics and bugs. In this document I will explain how most of the jump movement tech works, while trying to keep it as relevant as possible. A lot of this document was figured out while I was developing my bounce checking plugin as I wanted to use the in-game mechanics as a base for my plugin instead using a table of tested values. The rest was mostly figuring out how collision works, which is mainly relevant to the different types of wallbugs that exist.

2 Basics

Before some of the more interesting mechanics and funny bugs can be talked about, it is necessary to understand the basic concepts that Source engine games work with. The world that the game takes place in is 3-dimensional with an x -, y -, and z -axis to form a right-handed coordinate system. With this every point in the world can be represented by a coordinate (x, y, z) , where the x and y entries together control the horizontal position and the z entry controls the vertical offset with a greater value corresponding to a higher elevation. The game uses Hammer units when specifying the entries numerically, though they will just be referred to as “units” (and abbreviated with “u”) since no other length unit is used.

In this world there is the player that can move around by inputting commands to the game. The player also has an assigned coordinate to keep track of where they are at all times, which is called their origin. Everything about the player is described relative to this point as if anchored onto it. An example of that would be the player’s collision hull, which is used to handle collisions with the world and its other objects. Geometrically it is an axis-aligned bounding box, meaning it is a simple box that is aligned with the world axes, i.e. it does not rotate at all. The box itself is defined by specifying how far it should stretch out along each axis relative to the player’s origin. For the horizontal axes it expands 24 units out in the positive and negative directions, making the player 48 units wide in total. For the vertical part, the box does not get expanded down at all, while its top is put 82 or 62 units above the origin depending on whether the player is standing or crouching. The bounding box is therefore always anchored to the origin at its bottom center, no matter the situation. Then there is the viewpoint where the player looks from. Much like the bounding box, it is also placed relative to the player’s origin, though it is just a simple point instead of a box. The placement differs from class to class, but for both Soldier and Demoman it is located 68 units above the origin while standing and 45 units while ducking. It is possible to see the world position of the viewpoint by entering `cl_showpos 1` into the console, or `cl_showpos 2` to display the origin instead.

2.1 Vectors

This section is a short run-through of vectors for those who are either rusty or unfamiliar with them.

2.2 Game updates

For things to happen, the game needs to update the entities in the world to make them move and potentially do something interesting. As it is a computer that will do this, it will be impossible to keep up with real life physics where something happens in every instant of time. It is therefore necessary to simulate multiple seconds in a single game update to keep up with real time. Every game update is also not guaranteed to take the same time to complete, so to keep things consistent it is a good idea (from a game standpoint) to simulate a fixed amount of seconds every update with some compute time to spare. This kind of game update is called a frame in the Source engine, but it is purely logical and should not be confused with rendering frames that are displayed on a monitor. The amount of time a frame simulates is not fixed across the Source engine games, but in TF2 it is 0.015 seconds. The game therefore runs a new frame every 0.015 seconds, which gives an average of 66.666... game updates per second.

Given that there are multiple entities in the world, the game visits every applicable entity one-by-one to update them. It does this by going through a list created at the start of the frame, which does not necessarily use the same entity order throughout the frames. The update of an entity consists of updating its physics by simulating 0.015 seconds of time, unless it is a static entity, and going through any logic that is specific to that type of entity. That logic can be pretty much anything, such as a stationary teleport trigger that teleports entities that touch it to a target location, or maybe a gate that decides

when to start and stop opening by settings its speed, whereafter the movement itself gets handled by the physics.

Players are also entities that need to get updated, but unlike other entities, they need commands in order to do stuff. In offline single-player, the game can be sure to get a command every frame and the player can therefore get updated every frame along with every other entity. However when networking for multi-player enters the picture, it will not be guaranteed that the server receives a command from the client each frame. Trying to update the player every frame through something like interpolation would not work as that would end up clashing with the client that does get commands every frame. So the server ends up only updating the player whenever it receives commands from the client, leading to frames with zero or maybe even multiple updates in the event several commands were received during the execution of the previous frame. Because of this, players and entities they interact with cannot be expected to be synced up perfectly. Even if players got updated once every frame, like when playing offline (on a listen server), the ever-changing entity update order would still make interactions with other entities somewhat inconsistent.

2.2.1 Simulating physics

Trying to mimic 0.015 seconds of physics in a time shorter than that obviously requires a method that will not be 100% accurate. For games that is not really a big issue as it just needs to be “good enough”. Therefore it is possible to get away with using one of the simplest methods that assumes the velocity $\mathbf{v}$ of an object remains constant over a fixed time interval δ , which is just 0.015 seconds in this case. If an object is moving at the constant velocity $\mathbf{v}$ over a time δ , then its displacement over the time interval ends up being $\mathbf{v}\delta$. This way its position $\mathbf{r}$ will go through the change $\mathbf{r} \rightarrow \mathbf{r} + \mathbf{v}\delta$ every update.

While the assumption says the velocity is constant over a single update, it can easily change in between updates through an acceleration $\mathbf{a}$ or sudden collisions. The general idea for acceleration is that all continuous forces (gravity, drag, friction, etc.) that act on the object in idea gets averaged out over the timespan δ . So at the end of the update the velocity gets changed in a similar way with $\mathbf{v} \rightarrow \mathbf{v} + \mathbf{a}\delta$. Because the forces in question gets continuously applied every update, the acceleration might as well be applied at the start of the next update instead, such that an update has a nicer dependency order by doing $\mathbf{v} \rightarrow \mathbf{v} + \mathbf{a}\delta$ and then $\mathbf{r} \rightarrow \mathbf{r} + \mathbf{v}\delta$.

Because there will be many updates, it is a good idea to write the velocity and position as a function of time, meaning the above becomes $\mathbf{v}(t + \delta) = \mathbf{v}(t) + \mathbf{a}\delta$ and $\mathbf{r}(t + \delta) = \mathbf{r}(t) + \mathbf{v}(t + \delta)\delta$. The goal of simulating physics is to get a result where $\mathbf{v}(t)$ and $\mathbf{r}(t)$ are close enough to Newtonian mechanics that it can be passed off as realistic. With the method used here, the lower δ is, the more accurate the simulation becomes. In the limit where δ becomes zero, the simulation becomes fully accurate because $\mathbf{v}(t)$ and $\mathbf{r}(t)$ become continuous. To get an idea of how accurate the method is when using $\delta = 0.015$ seconds, the height from a simple jump as a function of time can be used. Gravity will be the only force present with its constant downwards acceleration of 800 u/s^2 in TF2, which together with the initial jump velocity $v(0) = 277 \text{ u/s}$ results in the plot below. Each dot represents the height after an update and the solid curve is where the dots should lie for the simulation to be fully accurate. As seen from the plot, the method does very well at matching the solid curve at the beginning, but after a while it begins to deviate more and more. Luckily the goal is not to exactly match the solid curve, but rather get something that is close enough to it, which the update interval of $\delta = 0.015$ seconds seems to be doing a fine job of.

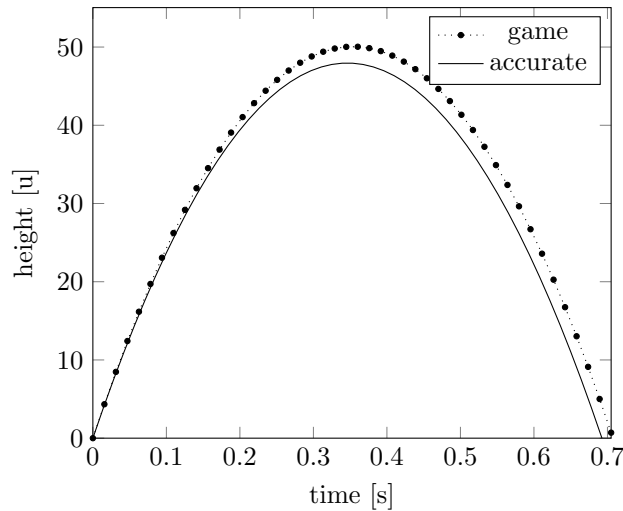


Figure 1

2.2.2 Player tick

The way a player moves around is the core of Source engine movement gamemodes and understanding it ultimately necessitates figuring out what happens to them every update. Given that it is so important, their update will be given its own name: a “tick”. This term is generally broadly used to talk about any game update that happens, but as mentioned, players do not necessarily update in sync with everything else. Most of the time this usage is no problem as player updates are usually the main focus, but it is important to remember the difference is there when playing a movement gamemode that relies on other entities. This is most prominent when trying to reach the achievable extremes of a rocket jump or when doing bounce setups with rockets that rely on the player getting the exact same knockback every time. In the first case the desired outcome is for the rocket to land exactly when the player is closest to the ground to receive the most amount of knockback. Rocket bounce setups where the player stands still are problem free, however as soon as a setup requires the player to move, the player and rocket will need to have the same number of updates for the setup to give the right knockback. In the end, no technique that requires the player and rocket to be synced to the frame/tick will ever be fully consistent.

Leaving that annoyance aside, a lot of things need to happen every tick as the player is the most integral part of a game. The main bullet points are ducking, jumping, acceleration, movement, grounding, touching triggers, and shooting. More specifically, with everything put in order, this is what the game does to a player every tick:

1. If stuck, attempt to find free space and skip to step 14,
2. If going up too fast, become airborne,
3. Handle ducking,
4. Apply half of gravity,
5. Handle jumping,
6. Cap velocity,
7. If on ground, zero out vertical velocity and apply friction,
8. Accelerate,
9. Movement and collisions,
10. Check for ground to stand on,
11. Apply other half of gravity,
12. If on ground, zero out vertical velocity,

13. Cap velocity,
14. Check for triggers to activate,
15. Update bounding box,
16. Shoot/detonate projectiles.

The steps in this list are not very descriptive because the list is only meant to serve as an overview to look back at. However giving a quick rundown of the most important steps would be beneficial to better understand how everything is tied together as a whole.

As a start, step 3 manages almost all the logic for ducking, with the exception of updating the bounding box itself which happens at step 15. Section 4 goes into the details and also explains how c-taps and headbugs work. Step 5 lets the player jump while they are on the ground, which is the topic of Section 3 that also covers how the bunnyhop prevention functions. Grounding the player is the job of step 10 with the implementation leading to sliding up ramps (or failing to), speedshots, bounces, and all the bouncehop variants. Everything about that is covered by Section 5. The acceleration of the player comes from gravity and their inputs. Gravity is split into steps 4 and 11, each contributing half of the total gravity for that update. Acceleration is applied following the method described in Section 2.2.1. Notice that gravity is negated while the player is grounded by steps 7 and 12. Section 7 has everything about how the player controls themselves via their inputs. The game also limits how much velocity the player can gain at steps 6 and 13 by capping the three components of the velocity to be within ± 3500 u/s. After the player's velocity has been updated, the game will begin moving the player in step 9. However the player will eventually collide with something, that usually being the ground or a wall. This is handled by a step by step process introduced in Section 8. Step 1 is also covered there and is meant to help the player if they get a bit stuck inside something, but it cannot do anything if they are in too deep. It is also responsible for stuck launches. How the game knows when the player actually collides with something is a lot more intricate and it is therefore split off into Section 9. It uncovers the reasons behind some of the interesting collision bugs that occur, such as wallbugs, pixelwalks, pixelsurfs, and texturebugs. The usage of projectiles is what Section 6 is all about. It has information on what happens when the player shoots a projectile to explain step 16, but also the knockback it applies to the player when it explodes. The reason behind the weird leniency of Market Gardener bunnyhops can also be found there.

3 Jumping

Jumping is probably the simplest action the player can do in terms of technical details, but it still has some intricacies to talk about. A jump takes place at step 5 from Section 2.2.2 and every time this part is reached, three things need to hold for the player to be able to jump: they are grounded, they are not crouched, and they started pressing jump on this tick. The first point is obvious, while the second one is probably there because jumping while crouched would look silly. The third point ensures that the player cannot just continuously input jump and expect to get a tick-perfect jump every time they land. This also makes scroll wheel bunnyhopping a 50/50 at best because the jump input needs to be released every other tick. The player will perform a jump if all the conditions are satisfied, and it follows these four ordered steps:

1. Cap speed,
2. Unground the player,
3. Apply jump velocity,
4. Apply half of gravity.

The last part is just a repeat of steps 4 and 11 of Section 2.2.2, while ungrounding the player means they should be considered airborne for the later steps in the tick.

Making the player physically jump is done by modifying their vertical velocity. Confusingly, there are two ways this is done. If the player is fully uncrouched, then the jump velocity of 289 u/s is added to the player's current vertical velocity. However, that is not true anymore if they are in the middle of ducking or unducking, which will for example happen if duck is inputted before jump in a crouchjump. In that case, their vertical velocity gets directly set to the jump velocity instead. There is no good reason

why this distinction should exist, but it does end up giving in a tiny difference in jump height. That is because step 4 of Section 2.2.2 changes the player's vertical velocity from zero to something slightly negative. As a consequence, adding the jump velocity to the vertical velocity gives a slightly lower jump than straight up overwriting its value. The height difference is only about 2 units, but it is there.

Lastly, the first step in the list is there as a measure to prevent players from keeping insane speeds by doing tick-perfect bunnyhops. This is achieved by capping player's speed such that it is no greater than 120% of their walking speed. The operation for this is quite simple: if their speed is above the cap, then their velocity is scaled down such that its magnitude is equal to the cap, otherwise nothing is done. Since the cap is larger than the player's walking speed, it is technically still possible to bunnyhop in TF2, but it is heavily nerfed. Another minuscule side effect of the implementation is that landing at high speeds will slightly increase the height of an uncrouched jump. That is because of the gravity from step 4 of Section 2.2.2 gets scaled down together with the horizontal speed, resulting in the player's vertical speed being less negative than normal. The height can also effectively be controlled by the player as a higher horizontal speed leads to more downscaling of the velocity components.

4 Crouching

The ability to crouch is a simple one on paper: the viewpoint smoothly lowers from 68 units to 45 units and the bounding box changes from 82 units to 62 units in height at the end of the action. It also works the other way around of course. For this to happen, it is necessary to know what state the player is currently in, which is accomplished with the crouch and ducking flags. The crouch flag controls if the player is fully crouched or not, i.e. it gets set when the player finishes ducking and gets unset when the player is standing up again. This flag is responsible for slowing the player down to a third of their walking speed and disallowing them to jump while they are crouching. Conversely, the ducking flag is set while the player is in the middle of ducking or unducking and is for example used to modify the jump velocity as discussed in Section 3. Ducking and unducking on the ground is not instant, so while the ducking flag is active, a timer will be running in the background. The timer is there to gradually lower or raise the player's viewpoint and ending the ducking process after a certain amount of time has passed. This is only relevant when the player is on the ground though, as crouching and uncrouching in the air is instant. It happens on the same tick the player changes their duck input, so no kind of transition takes place, i.e. the ducking flag never gets set. Crouching in the air also raises the player's legs rather than lowering their head, which is useful for reaching higher ledges. More specifically, instead of just setting the bounding box height to 62 units, the player's origin also gets raised up by the 20 unit height difference, creating the illusion of them pulling their legs up. The reverse is also true for uncrouching in the air.

4.1 Crouch-tap jump

A technique for getting more power out of a grounded rocket jump is always useful to have. One such technique is crouch-tap jumping, or c-tapping for short. It is performed by initiating a duck, then cancelling the duck before the player becomes crouched. After that, while the player is still unducking, jump and shoot. The jump induces a c-tap that makes the player do a crouchjump that starts closer to the ground than normal. That consequently leads to more knockback as they will be closer the explosion happening beneath them. Getting a lower crouchjump from this sequence of inputs is pretty unexpected and does not really make any sense, but the reason for that is actually very simple: c-taps are a bug.

The bug relies on the extra restriction that uncrouching has that crouching does not. The player is able to become crouched no matter what position they are in because their bounding box never goes outside the space it was already occupying. The same cannot be said about uncrouching as the bounding box needs extra room to expand into to fit the player's uncrouched volume. A check therefore happens whenever the player tries to uncrouch to ensure its possible to do so. Furthermore, to aid with crouching under something while not holding duck, the player also constantly gets set into the fully crouched state if the check thinks it is impossible to uncrouch, i.e. setting the crouch flag, clearing the ducking flag, and resetting the unducking timer. This part is only meant to execute if the player is trying to uncrouch from a crouched position, but it is possible to trick the system. This is where the first part of the c-tap input sequence comes into the picture. If the player is standing on the ground, it is possible to initiate a duck and then cancel it to enter an unducking state while uncrouched. This in itself does not lead to any problems as there is enough space to unduck into given that the player is already standing upright. It is however possible to create a problem from this state by performing a jump that subsequently puts

the player into an airborne state. After that, when step 3 of Section 2.2.2 is reached on the next tick, the ducking flag will still be active, which is not normally possible while in the air where all crouching actions are instant. Since the player is still not inputting duck, they end up getting mistaken as being in the middle of unducking, essentially continuing what they were already doing on the ground. This means the uncrouch check is going to happen even if they are currently uncrouched in the air. Since the player's legs extend when uncrouching in the air, the check naturally looks 20 units below the player for available space. It is now the trouble starts.

The player has only spent one tick in the air so far, nowhere near enough to cover the 20 units for the check to accept an uncrouch. The player is now considered to be trying to uncrouch without the space to do so. This activates the mechanism that forces the player into a crouched state when they are trying to uncrouch, but are unable to do so. However this mechanism is only intended to be run while the player is already crouched, so their origin position will remain untouched, as opposed to normal where the origin would be moved 20 units up when crouching mid-air. The end result is that only the bounding box changes into its crouched form while the player's feet stay put. The player is therefore 20 units closer to the ground than a normal crouchjump, while still being able to enjoy the increased knockback from being in a crouched state. After rising 20 units up in the air, the player will automatically uncrouch unless they pressed the duck input again after jumping. To more easily conceptualize the result of a c-tap, imagine if it was possible to jump while fully crouched. A c-tap would be identical to such a jump, with the exception of the very first tick where the player is still uncrouched.

The input sequence described above is the normal way to induce a c-tap, but there is actually one more way to it. The method abuses the fact that the ducking flag does not distinguish between ducking and unducking. It is therefore possible to perform a crouchjump and then release the duck input on the very next tick. This will setup the exact same scenario as before where the ducking flag is active and the player is not inputting duck. Being able to do a c-tap in a different way is all well and good, but it seems needlessly hard compared to the normal method that does the exact same thing. There is one benefit to get from it however. With some fast fingers, it is possible to do a c-tap bunnyhop using this method by inputting duck and jump on the exact same tick the player lands to perform the crouchjump, followed by the duck release right after. A c-tap bunnyhop therefore becomes a difficult single-tick input technique that requires tick-perfect timing. There is technically a way to remove the challenge of timing the duck release, but it requires shallow water. That is because water forces any airborne player to uncrouch while they are within it. So by simply doing a crouchjump while standing in water, the player will end up doing a c-tap whether they want to or not because the duck input will be released for them.

4.2 Headbug

The discussion around the ducking mechanics has been centered around step 3 from Section 2.2.2, but it is important to note that crouching and uncrouching is actually split into two steps: step 3 that handles all the ducking logic, and step 15, which is responsible for updating the bounding box in accordance to the crouch flag. This means that every time the player crouches in the air, there will be a small period where the player's origin has been raised up, but the bounding box is still standing tall with its 82 unit height. It is possible to exploit this by noticing that trigger handling happens at step 14, right before the bounding box gets updated. So if there happens to be a ceiling that is less than 20 units thick and has a trigger located right above it, the action of crouching makes it possible for the offset standing volume to poke through the ceiling, consequently activating the trigger. The act of touching a trigger through a ceiling this way is referred to as a headbug and it can be used for skipping ahead to a later jump or making an early exit on a vertical jump with a teleport door. Some jump maps even intentionally include headbugs as a mechanic for a jump.

5 Grounding

For the game to know if it should apply air or ground movement to the player it needs to know if the player should be considered on the ground or not. To know the current state of the player the game uses a variable that will exclusively be null if the player is airborne (otherwise points to the entity the player is currently standing on, which is most commonly the world entity). One place where this variable is changed is when the player jumps, but to handle landing again, which should set the player to a grounded state, it uses routine that takes a few factors into consideration. If the player is moving up faster than 250 units per second it considers the player airborne as sets their state appropriately. This is what allows sliding up ramps by having enough speed even if they are possible to stand on. The next thing it checks

is if there is any ground below the player that can be stood on, and in the case there is, the player will become grounded. The ground needs to be at most 2 units below the player for it to be counted and it is considered standable only if the vertical component of its normal is greater than or equal to 0.7, which is the same as if its angle is below approximately 45.5 degrees. In the case the player was already grounded before this routine, it will move the player down to the ground instead of leaving them within some 2 unit range of the ground.

5.1 Rampbug

A common annoyance when jumping is hitting a standable ramp with a good amount of speed and stopping completely on it. This is referred to as a rampbug, which should not be confused with rampbugs that sometimes happen while surfing as they happen for a completely different reason. What would normally happen is that the velocity would be projected along the ramp when hit, which gives an upwards velocity greater than 250 units per second, such that the player would not be grounded. The reason all speed is suddenly lost is because the ramp was not actually hit. Instead the tick ended with the player being within 2 units above the ramp while falling towards it and they therefore get grounded (in bhop this is actually desired as the ramp can then be jumped off of). On the next tick the player will then either be capped to their walking speed or 120% of their walking speed if they decide jump.

5.2 Speedshot

Speedshots are a nice way to keep or add horizontal speed because, unlike pogo, it does not require the player to knock themselves upwards to avoid hitting the ground. Instead speedshots work off of the fact that walk speed capping will only be applied the tick after the player has been grounded, so if they get knockback from an explosive just after they hit the ground, it is possible for them to get enough vertical speed (250 u/s) to become airborne again via step 2 of Section 2.2.2. As neither friction nor capping is performed after becoming airborne again, the speedshot will simply add the knockback to the velocity the player already had, except it is also possible to gain a good bit of height from it as the vertical velocity was already zeroed out from the collision with the ground.

5.2.1 Velocity dependency

5.3 Bounce

A common gimmick in jump maps is something called bouncing where the player is able to either jump on or push themselves away from a surface without activating a teleport trigger that lies on top of it. The normal variants of bounces are uncrouched/crouched bounces where the player times the explosion such that it happens at the same time they land, or bouncehops where the player simply jumps when they land. Using the jump to perform a rocket jump is referred to as a powerbounce. Bounces work much like speedshots, except there is now a trigger that needs to be avoided. As such, the player is not allowed to collide with the ground, but instead needs to make use of the 2 units of leeway that is used for finding a standable surface. This is not quite enough however, as the trigger has its own volume that the player needs to be outside of. In the case where the trigger is 2 units or higher, it becomes impossible to get grounded without touching the trigger and bounces will therefore not be doable. Luckily this is not a big problem as 1 unit high teleport triggers are very common in jump maps.

As mentioned, bounces work a lot like speedshots, but for bounces the important part is not preserving speed, but rather avoiding getting teleported. The reason bounces are so precise to perform is because the player will be moved down to the ground on the next tick, which was briefly mentioned in the general grounding explanation. This means it is necessary for the player to unground themselves before the game checks for standable ground again. Looking at Section 2.2.2 there are two options to achieve this, either become airborne by getting knocked upwards right after getting grounded (uncrouched/crouched bounce), or simply jumping (bouncehop/powerbounce) which is possible because the player is currently grounded.

5.3.1 Jumpbug and Glidestep

In the case the teleport trigger is too high to do the bounce methods mentioned above, then jumpbugs might be a viable alternative as they can allow the player to completely bypass the trigger. Jumpbugs are performed by starting crouched in the air and then simultaneously uncrouching and jumping at a height

where the player would have been grounded had they not been crouched. The fact that uncrouching allows for an immediate jump to can be attributed to the grounding check done after uncrouching or crouching fully, which is mentioned in Section 4. So because uncrouching is performed before jumping (refer to Section 2.2.2), it is possible to get grounded and jump in the same tick. Jumpbugs are therefore also a useful technique to avoid all fall damage as the player both starts and ends the tick airborne. The last part to jumpbugs that allows the player to actually avoid the teleport triggers is simply going a bit up from the jump. As triggers are only checked for after these three parts, it is possible for the player to uncrouch and get grounded, potentially leaving them inside the trigger, but then immediately getting moved out of it again, allowing them to escape the trigger before it gets activated. This is of course not possible if the trigger is too high as the velocity from the jump only moves the player up by about 4 units.

Another use of the mechanics behind jumpbugs is to rapidly ascend a staircase that can only be scaled by jumping. This technique is called glidestepping and the reason it is so fast is because the player is able to land on the next step while going up instead of falling down. It relies on the game constantly checking for available space for the player to uncrouch. In the case there is, the player will get uncrouched, meaning a jumpbug can be performed if the player ends up close enough to the ground and also times their jump properly. The possible step heights are therefore heavily restricted since the player will in most cases not get grounded after they uncrouch simply because they are not within 2 units of the ground. Glidesteps also have another part to their execution, namely jumping and getting above the step while crouched, then stopping inputting duck before missing the window to automatically uncrouch right above the step. So the whole chain of inputs the player has to do is go towards the step, jump, crouch for a short time, and then jump again. These actions can then be repeated for each step to quickly climb the whole staircase assuming that the jumping from the previous step allows for a jumpbug on the next one (the player will not jumpbug directly on the ground, but a bit above it).

5.3.2 Viability

There is a difference between a technique existing and it being viable as a game mechanic. In the case of bounces, the window the player needs to be within is incredibly small, so it is unfeasible to do it a whim when landing from something like a rocket jump. Therefore it is almost necessary to find consistent setups for bounces. For example simply jumping off a ledge located above the ground that the player wants to bounce on could coincidentally make them enter the window where a bounce is possible. This creates a consistent setup that will always work as long as the player hits the timing for the bounce itself. Because brute forcing all the different possible setups to see if they work would be tiresome, most jump servers have plugins made specifically to check what kind of setups are possible (there are also websites that give setups using an inputted starting height). Consistent setups are not always necessary though. If the player is falling very slowly, like while pogoing, there is a good chance that a bounce can be performed as the lower speed makes the bounce window less likely to be missed. The mention of chance is only in the eyes of the player because they have no good way of knowing if they will miss the bounce window during their current fall without the use of a tool that constantly updates.

5.3.3 Chaining bounces

There are a few other techniques that should be mentioned, especially when trying to chain bounces, where it is mostly assumed that the trigger is 1 unit high. The first one is simply crouching and jumping at the same time to perform a crouchjump, with the only difference to bouncehops being the jump velocity. The jump velocity plays an important factor when bounces are to be performed in succession as it controls if the player lands inside the 2 unit window or not when attempting the next bounce. Most of the time this would be jumping down to a surface lower than the current one, but it is also possible to bouncehop on the same surface twice if the first bouncehop is performed at a proper distance above the ground. It is not possible to chain three bouncehops if all jumps result in the same vertical velocity after the jumps, but as mentioned in Section 3, a normal jump actually depends a bit on the player's horizontal speed. This actually makes it possible to chain three bouncehops on the same surface on the condition that the player's horizontal speed changes the jump such that the next bouncehop is correctly set up for a double bouncehop. A way to jump even more times on the same surface is to perform c-tap jumps (see Section 4.1) as the player's position is only lowered by 0.005 units when they land again, making it possible to chain them many times. However their timing is incredibly strict, making it unviable for humans to perform multiple of them in a row. This is because the player needs to get the ducking flag and jump on the same tick to respectively initiate a c-tap and avoid the teleport trigger. This is done

by ducking and jumping on the same tick, but then to finalize the c-tap the player needs to release duck on the very next tick, otherwise the player would just perform a crouchjump. Additionally, the player needs to keep themselves crouched for a bit after this as to not get caught by the teleport trigger when the player automatically tries to uncrouch from going high enough above the ground.

6 Projectiles and Knockback

It is now time to go into details about rockets and stickies which form the basis of jumping. That of course consists of multiple parts, namely

- Where the projectiles are spawned when fired,
- What course the projectiles take after being fired,
- When the projectiles collide with the world and explode,
- How the projectiles apply blast knockback to the player.

Starting from the top, whenever a projectile is created, it will be done with a fixed offset relative to the player's viewpoint. This changes depending on the weapon that is used, though some values are shared across the lineup. The offsets exist so the projectiles are spawned around the viewmodel itself, but they also help to prevent the player's view from getting blocked whenever a grenade, sticky, or rocket is fired. Before getting into the actual numbers, it is important to understand what is meant by a relative offset. The world has its x, y, and z axes, but the player can also be given their own coordinate system using their viewpoint and aim direction as a base. That makes it easy to place a projectile relative to where the player is aiming because it is possible to just specify how far forward, right, and up the projectile should be placed. Since this is a proper coordinate system, the values along these view axes can also be negative. For example, using flipped viewmodels simply negates the value of the right offset to make it spawn on the other side of the screen. Another point of confusion could be the interpretation of "up". As this is meant to be a coordinate system where all axes are perpendicular, it means up is not along the z-axis of the world, but rather the direction that goes directly above the head. So looking straight up or down would make the up direction point horizontally along the world (actually only possible to look 89 degrees up and down, but close enough).

The grenade and sticky launcher have the simplest offsets of all the relevant launchers because they are both fixed at (16, 8, -6). The order of the listed directions are forward, right, and up, technically making it a left-handed coordinate system as opposed to the conventional right-handed coordinate system the world uses with its x, y, and z axes. So a grenade and sticky will be spawned 16 units forward, 8 units right, and 6 units down relative to the player's viewpoint. The various rocket launchers have a bit more to them in terms of offsets. First, they all share the same forward offset of 23.5 units, making the rocket spawn just inside the player's bounding box when they are looking straightforward. They also share the up offset, but it comes with a condition. It gets set to -3 units while the player is standing and 8 units while they are crouched (Figure 2a). The last axis is the right one, which is also the most discussed. The right offset differs per rocket launcher (Figure 2b), though most of them share the stock offset of 12 units. Then there is the Original with its 0 unit offset, whose existence have stirred many highly intellectual debates. Lastly there is the lesser known third offset of 8, which is what the Cow Mangler 5000 uses. Table 1 can be used for a more concise list of the values.

The second part of the projectile list is where it starts to get intricate. The approach taken for directing the projectiles forward is done in one of two ways. For the grenade and sticky launcher, it is done by simply giving the projectile an independent forward, right, and up velocity. The right and up velocities are picked randomly within the ranges -10-10 u/s and 190-210 u/s respectively. So the grenade or sticky will be shot relatively straight from its firing point, but with an upwards velocity to create an arc. The forward velocity differs for grenades and stickies. A grenade is always fired forward with a speed of 1200 u/s, while a sticky can be within the range 900-2400 u/s which is fully controlled by how much the sticky launcher has been charged up. After the initial velocity has been given, the grenade or sticky will fall while drag is applied to it.

The other approach, that is adopted by the big family of rocket launchers, uses the location that the player is currently looking at to direct the rockets. The general idea is to send the rocket directly to the place under their crosshair at a constant 1100 u/s such that it will hit whatever target might be there (Figure 2c). However it is not as straightforward as that. In actuality, the game only looks 2000

units ahead for something to hit. If the player is aiming at something further away than that, then the rocket will not be given a proper target location. Instead it will fly towards the point 2000 units in front of the player's viewpoint, exactly where the game stopped searching for something. For rocket launchers with a right offset, this means the rocket will start on the right side, cross the player's crosshair at the empty location 2000 units away, and then continue onto the left side of the crosshair until it hits something (Figure 2d). This effect also happens for the up spawn offset, but it is not noticed as often. The method also has a caveat for objects that are close to the player because the rocket would come out at an unrealistic angle if it went straight for the crosshair. The game solves this by also aiming rockets for the 2000 unit mark whenever an object is closer than 200 units away from the player's viewpoint (Figure 2e). This way the rocket will go more or less straightforward because its end point is so far away, which ends up looking a lot better than the alternative. As a side effect for rocket jumping, the rockets will be mostly going straight from its spawn offset, unless syncs or pre-fires are involved in some capacity. For the Beggar's Bazooka, the spawn offset and the travel direction follow the exact same steps as the stock launcher, but with the player's aim effectively being modified just before that. The aim used for the rocket will be pointing in a random direction within an upside down pyramid that expands at a 3 degree decline away from the player's viewpoint. If the weapon gets overloaded, then the rockets will explode directly at the spawn offset.

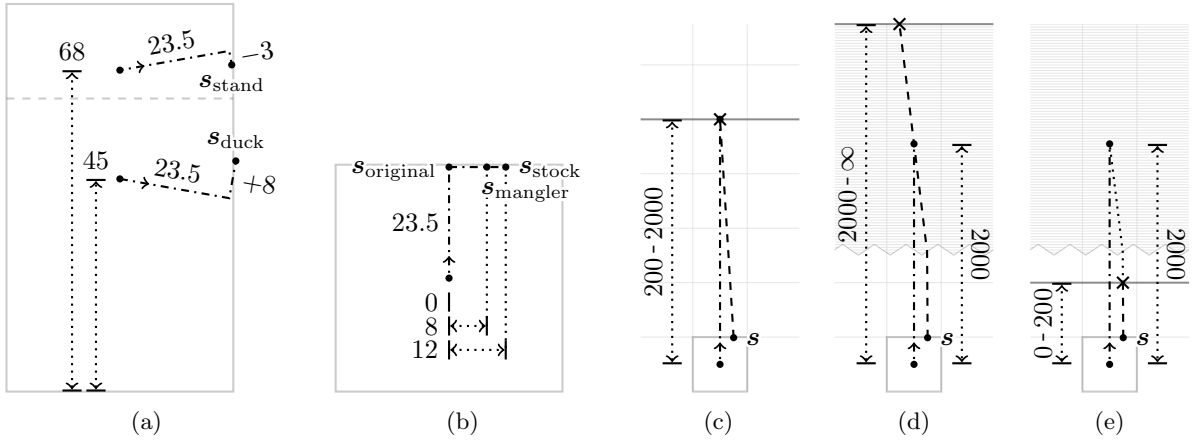


Figure 2

At some point the projectiles will collide with something. Both grenades and stickies collide with the world in a way that generally matches their shape via the VPhysics system. A grenade will either bounce or roll when this happens, as opposed to a sticky that sticks to whatever it hit by getting its physics updates suspended. After 2.145 seconds a grenade will automatically explode (actually a few frames later than this for reasons discussed in Section 6.1), while a sticky needs to wait 0.8 seconds before it can be manually detonated. When a grenade or sticky explodes, a check will begin 8 units above it that looks 32 units directly down for any kind of surface. If something is found, the projectile will get teleported to a position 1 unit away from that surface, otherwise it will be moved to the start of the check. This means grenades and stickies will be moved a bit down just before exploding if they are on the ground because of their larger collision radius. Meanwhile being in the air or mounted to a wall make them explode 8 units further up. A rocket is far simpler as it is just a point entity that flies in a straight line. Its model does therefore not represent its collision at all and it is able to slip through even the most narrow crevices with the right angle. When a rocket finally collides with a surface, it will be moved 1 unit away from whatever it hit and finally explode.

When a projectile explodes it will apply blast damage originating from its origin to anybody nearby. To make jumping more consistent across the various unlocks, the projectiles use the same base damage and explosive radius for self damage. The base damage for grenades, stickies, and rockets are 100, 120, and 90 respectively, while the explosive radii are 146, 146, and 121 units following the same order. The Cow Mangler 5000 has the small quirk that its charged shot increases the radius by 33% to 160.93 units, but in turn slows the player down to scoped speed at 80 u/s while its charging.

To apply the blast self damage to the player, it is first necessary to know if they are close enough to the explosion. This is done by finding the closest point on the player's bounding box and seeing if that is within the explosive radius. In the case it is, the next step is to figure out the distance to center of the player's bounding box and the center of their feet. The distance that is smallest will be used to calculate

the damage falloff from the explosion, which aids in getting more power out of blast jumps done from the ground. The actual self damage done goes through multiple steps, but the initial damage that is based on the distance is

$$D_{\text{dist}} = D_{\text{base}} \left(1 - \frac{1}{2} \min \left(\frac{d}{r}, 1 \right) \right). \quad (1)$$

Here D_{base} is the base damage, d is the selected distance to the explosion, r is the explosive radius, and $1/2$ is the falloff, which caps the minimum damage taken to half of the base damage. This is just a map that returns D_{base} when $d = 0$ and $\frac{1}{2}D_{\text{base}}$ when $d \geq r$, with everything in between being linearly interpolated. It can be thought of as inverting and scaling the interval $(0, r)$ onto $(\frac{1}{2}D_{\text{base}}, D_{\text{base}})$.

To make jumping (and explosive spamming) more viable in actual games, both Demoman and Soldier get their own unique damage reduction. Whenever a Demoman gets hit by his own grenade or sticky, he will only receive 75% of the damage D_{dist} . For Soldier, the damage reduction is not based on the projectile, but rather his current state. If he is neither on the ground nor in water, then he will only take 60% of D_{dist} from his own explosions. So basically whenever the player executes a proper rocket jump. This modified damage D_{mod} will be the actual self damage the player will take from their own explosion.

The final part consists of turning D_{mod} into actual knockback. The direction the player will be pushed towards is found by using the point 10 units below the origin of the explosive and getting the angle to the player's center. In practice this is done by computing the difference of the two points and normalizing the resulting vector into $\hat{u}$ as that is easier to work with. The 10 unit offset exists to make it easier for players to get send upwards from damage. The strength of the knockback is in idea done by applying an impulse to the player with the equation $p = mu$ as would "realistically" happen, but it is of course not as simple as that in the end. In the equation, p is the impulse the explosive applies to the player, m is their mass, and u is the resulting knockback velocity. Therefore the strength of the knockback is $u = p/m$.

The formula for the player's mass assumes they always have the same density $\rho = V_{\text{stand}}^{-1}$ such that $m = \rho V = V/V_{\text{stand}}$. So the mass is just the ratio of volume of the player's bounding box and the volume of the standing bounding box. As u is inversely proportional to m , having a smaller bounding box ends up yielding more knockback, leading to the practice of crouching while blast jumping. There is an inaccuracy in that formula though, because in some update the height of the crouched bounding box got changed from 55 units to the 62 units of today. To not mess up rocket jumping, a small check was added such that whenever the bounding box is equal to the crouched bounding box, a 55 unit high bounding box is used instead for the mass calculation. Therefore the two most important masses become $m_{\text{stand}} = (48 \cdot 48 \cdot 82)/V_{\text{stand}} = 1$ and $m_{\text{duck}} = (48 \cdot 48 \cdot 55)/V_{\text{stand}} = 55/82 \approx 0.67$.

The impulse is different for Demoman and Soldier because it uses a knockback modifier to the damage through $p = KD_{\text{mod}}$. For Demoman, K is always 9, contrary to Soldier where it is 5 when he is on the ground and 10 when he is not. The distinction between being in the air and not on the ground is very important for doing rocket jumps in water. That is because if Soldier rocket jumps in water, he will not get the damage scaled down to 60%, but he will still have a knockback modifier of 10. Being in water will therefore amplify the player's rocket jumps because of the extra damage taken. If the player does a rocket jump in shallow water, it is even possible for them to hit the knockback cap of 1000 u/s, the maximum possible value u can take.

By combining everything up until this point, the knockback can now be computed using $\Delta \mathbf{v} = u\hat{u}$, which is directly added onto the player's current velocity. Below is a quick summary for most of the equations and values introduced.

$$\Delta \mathbf{v} = \hat{u}u = \hat{u} \min(p/m, 1000) = \hat{u} \min \left(\frac{KD_{\text{mod}}}{\tilde{V}/V_{\text{stand}}}, 1000 \right), \quad (2a)$$

$$m = \begin{cases} 1 & \text{standing,} \\ 55/82 & \text{crouched,} \end{cases} \quad (2b)$$

$$K = \begin{cases} 9 & \text{Demoman,} \\ 5 & \text{Soldier on ground,} \\ 10 & \text{Soldier not on ground,} \end{cases} \quad (2c)$$

$$D_{\text{mod}} = D_{\text{dist}} \cdot \begin{cases} 75\% & \text{Demoman,} \\ 100\% & \text{Soldier not in air,} \\ 60\% & \text{Soldier in air,} \end{cases} \quad \text{where } D_{\text{dist}} = \text{Equation (1)}. \quad (2d)$$

6.1 Market Gardener bunnyhop

A fun alternative use for rocket jumping that does not involve playing on jump maps is getting the drop on people with the Market Gardener in a PVP setting. One of the unintended quirks of the Market Gardener is its ability to keep its crits by bunnyhopping. Weirdly enough, the trick does not share the single tick nature of speed preserving bunnyhops and bouncehops. Instead, the timing window fluctuates and makes Market Gardener bunnyhops range from easy to hit to being outright impossible. The reason behind it ends up being a bit silly. The player gets marked as blast jumping if they are knocked into the air by an explosion, which is what the Market Gardener uses to check if it should crit. So for the crits to get taken away when the player lands, this blast jump state needs to be removed.

This is done through a regular routine that checks if the player is not airborne anymore, in which case the blast jump state gets cleared and the player will lose their crits. Every time this check is done, the next routine will get scheduled for 0 seconds into the future. Because a routine can only run a single time per tick, this essentially means it should be scheduled for the tick right after. That was the intention at least. Before the player tick is run, the current server time actually gets modified a little bit, which according to comments from the Source SDK helps improve client-side interpolation. The change tries to have the time be around 60 ms in the future, or whatever `sv_clockcorrection_msecs` is set to. That value is also used as the limit for how much the modified time can deviate from the planned time. Since 60 ms is equal to 4 ticks in TF2, that means the time is meant to be around 4 ticks into the future, but in reality it can be anything between 0 and 7 ticks depending on the player's connection.

The change in time kind of messes up scheduling, as adding 0 seconds to the current time now results in a time that will usually be a few ticks into the future relative to the actual server time. While the routine for taking away crits is executed during the player tick where the time is modified, not all values related to the server time is changed, and as luck would have it, the time that is used to check against the scheduled time never gets touched. As a consequence, the routine generally only runs every few ticks instead of every tick. It therefore becomes possible for the player to land on the ground and jump up before the routine is run again. This is what makes it possible to retain crits though a bunnyhop. A few ticks might sound like a lot, but the player will on average land when half of the scheduled time has already passed, effectively halving the time to Market Gardener bunnyhop. The player might also get so unlucky that the modified time is only offset by 0 or 1 ticks, as the routine will then run every tick and void the crits before the player even has a chance to jump. There is also the possibility that they might land the tick before the routine is run, which leads to the same issue. So all in all, Market Gardener bunnyhops generally have more lenient timing windows, but they can also be impossible to perform in some cases.

7 Acceleration

This section will deal with pretty much everything else that changes the player's velocity except for collisions. The main contributors are gravity, acceleration, friction, water, and velocity capping. Gravity and max velocity capping were described in Section 2.2.1 and 2.2.2 so it will not be repeated here, but a main point to take away from it is that the change in velocity is mainly calculated by multiplying the relevant acceleration with the tick interval to the new velocity. Starting off with friction that is only applied while on the ground, to calculate the acceleration it uses the coefficient of friction μ and the player's current speed v by multiplying them together. However there is a catch, if the speed is below the stopping speed v_{stop} , then that will be used instead when calculating the acceleration to prevent the player from never fully stopping. So the acceleration can be written as $a = \mu \min(v, v_{\text{stop}})$ which is then multiplied by the tick interval to get the speed loss that is used to decrease the player's speed.

To be able to move around the world the player is able to control some of their horizontal acceleration themselves via their inputs. The relevant factors that pop up are their max uncrouched walking speed v_{walk} , wish velocity $\mathbf{v}_{\text{wish}}$, the speed cap for acceleration v_{cap} , and the acceleration factor a_{factor} . The wish velocity is what the player controls with their inputs. It stores how fast the player is trying to go (v_{wish}) and in what direction ($\hat{\mathbf{v}}_{\text{wish}}$). The wish speed is usually just v_{walk} , which it cannot exceed, but it can be lowered by either crouching down or by using a controller. Lastly v_{cap} and a_{factor} are respectively v_{wish} and a_{walk} if the player is on the ground or $\max(v_{\text{wish}}, v_{\text{air}})$ and a_{air} if they are airborne.

From this the acceleration can easily be found with $\mathbf{a} = \mathbf{v}_{\text{wish}} a_{\text{factor}}$ and the change in velocity would therefore be $\Delta \mathbf{v} = \mathbf{a} \delta$. However to prevent the player from accelerating to speeds greater than their wish speed v_{wish} it is necessary to somehow cap the change in speed along their wish direction $\hat{\mathbf{v}}_{\text{wish}}$. This is done by first finding the player current speed along $\hat{\mathbf{v}}_{\text{wish}}$ and subtract it from v_{cap} , which is done with

the calculation $v_{\text{cap}} - \mathbf{v} \cdot \hat{\mathbf{v}}_{\text{wish}}$ where $\mathbf{v}$ is the player's current speed. The speed of $\Delta \mathbf{v}$ is then capped to be below this value, or in the case the value is negative no change in velocity will occur. This ensures that the velocity added will not make the player's speed greater than v_{cap} along the wish direction. The velocity change with all the conditions can be written as

$$\Delta \mathbf{v} = \hat{\mathbf{v}}_{\text{wish}} \min(v_{\text{wish}} a_{\text{factor}} \delta, \max(0, v_{\text{cap}} - \mathbf{v} \cdot \hat{\mathbf{v}}_{\text{wish}})). \quad (3)$$

The last that needs to be done is to cap the newly achieved velocity horizontally to v_{walk} if the player is on the ground and decrease their backwards speed by 10% in the case they are backtreading.

7.1 Strafing

It is very easy to control the path of the player while walking by simply looking towards the desired destination and pressing forward. This is because the player's top speed is the same as the speed used to cap the magnitude of the velocity change, meaning the cap will always be positive if the player is trying to change their direction, which in turn lets the player accelerate towards where they are aiming with no problems. The opposite is true when the player is airborne. Most of the time the player will have more speed than the low air speed cap and it is therefore impossible to navigate the air in a meaningful way by applying the same method that is used on the ground.

Air strafing is a technique that allows the player to turn in the air without losing their horizontal speed, but it requires less haphazard inputs. It is done by inputting a direction perpendicular to the player's current velocity and turning the camera such that the player is always looking along the same direction compared to their velocity. Using the most common strafing technique as an example, if the player wants to turn right in the air, they input right and turn their view at a rate where they will always be looking in the direction they are going. It works of the fact that the acceleration speed cap can be avoided by inputting a direction perpendicular to the current velocity such that $\mathbf{v} \cdot \hat{\mathbf{v}}_{\text{wish}} = 0$, meaning that the player's current speed does not hinder their acceleration at all along the inputted direction. As the velocity has been skewed along the input direction, it is necessary for the player to aim along this new velocity to constantly perform a perpendicular input, allowing for a continuous turn mid-air. Air strafing also serves as a mean to gain speed while in the air as the velocity change is perpendicular to the current velocity meaning $|\mathbf{v} + \Delta \mathbf{v}| > |\mathbf{v}|$.

7.1.1 Wall-hugging

In the case the player is in a tight space where they can only crouchwalk, like a vent, it is possible to increase their speed by walking into a nearby wall at an angle and simply inputting forward. The reason this is not possible while standing is because the player cannot be faster than walking speed while on the ground. The wall-hug method is very similar to increasing the player's speed by air strafing, where the player is constantly keeping their aim the same relative to their current velocity. However instead of having the player adjust their aim, the wall instead adjusts their velocity to always go in the same direction from the collision. This way the projected velocity $\mathbf{v} \cdot \hat{\mathbf{v}}_{\text{wish}}$ is less than v_{cap} , allowing the player to accelerate towards the wall and gain speed perpendicular to it, but this constant speed gain will not last forever as friction will at some point be on par with the acceleration. This technique is also possible to pull off in the air and it will instead be $v_{\text{cap}} - \mathbf{v} \cdot \hat{\mathbf{v}}_{\text{wish}}$ that limits the possible speed gain each tick as the player gets faster and faster.

8 Movement

After the player's velocity has been affected by gravity, friction, and their inputs, the next step is to move them around according to this velocity. In the case there is nothing in the way to stop them, their new position is simply $\mathbf{r}_{\text{new}} = \mathbf{r} + \mathbf{v}\delta$ where δ is the tick interval, $\mathbf{r}$ is the player's current position, and $\mathbf{v}$ is their velocity. If the player does suddenly find themselves with something in front of them, they need to be affected by it whenever they collide with it.

All collision detection is done by using a thing called a trace. There are two types of traces, the first one is an unswept trace that is used to check if the player would get stuck at a specified position. The other type is a swept trace and it is the one that is used for collision detection. It functions by using a start and end position to figure out how far from the start the player would go before hitting something. It gives back four useful items: the position the player would be at after a collision, the fraction of the distance the player would travel compared to the end position given to the trace, the normal of the

collision surface, and if the player started out being stuck. If the player would not collide with anything, then the trace just gives back the end position it was given and a fraction of one.

By applying a swept trace it would now be easy to figure out if the player would hit a surface and end the movement for this tick by placing the player at the collision point and modifying their velocity accordingly. However, in idea, the time it took the player to reach the collision point would be less than the tick interval δ because the end position given to the trace was further away. It is therefore important to continue the movement in a tick even after the player has collided with something. The game does this by keeping track of the remaining time t after each collision. The remaining time starts at $t = \delta$ and decreases with every collision by subtracting how long it took to reach the surface from the current remaining time. In the end, if there is nothing in the way of the player after one or more collisions, all of the remaining time will be used up when moving the player by $\mathbf{v}t$.

To go further into detail about how a collision is simulated, assume t is the time left, $\mathbf{r}$ is the player's current position, and $\mathbf{v}$ is their velocity. The game will then start a swept trace at $\mathbf{r}_{\text{start}} = \mathbf{r}$ and set the end position to $\mathbf{r}_{\text{end}} = \mathbf{r} + \mathbf{v}t$. If there is a surface in the way the trace gives back the collision point $\mathbf{r}_{\text{col}}$ and the spanned fraction $f = |\mathbf{r}_{\text{col}} - \mathbf{r}_{\text{start}}|/|\mathbf{r}_{\text{end}} - \mathbf{r}_{\text{start}}|$. From here the player is moved to $\mathbf{r}_{\text{col}}$ and the time left is subtracted by ft as that is the time it took to hit the surface. Lastly the velocity is changed such that it is no longer pointing into the surface. In practice this is done by projecting $\mathbf{v}$ onto the surface normal $\hat{\mathbf{n}}$ and removing that part from $\mathbf{v}$. The velocity after the collision therefore becomes $\mathbf{v}_{\text{col}} = \mathbf{v} - (\mathbf{v} \cdot \hat{\mathbf{n}})\hat{\mathbf{n}}$ since $\mathbf{v} \cdot \hat{\mathbf{n}}$ is the speed at which the player is going directly into the surface.

To sum all of this up in a simple algorithm, set $n = 0$, $\mathbf{r}_0 = \mathbf{r}$, $\mathbf{v}_0 = \mathbf{v}$, and $t_0 = \delta$ as starting parameters and then follow the given steps:

1. Set $\mathbf{r}_{n+1}^{\text{start}} = \mathbf{r}_n$ and $\mathbf{r}_{n+1}^{\text{end}} = \mathbf{r}_n + \mathbf{v}_n t_n$ to get collision/end point $\mathbf{r}_{n+1}$ and fraction f_{n+1} from swept trace,
2. If player is reported stuck using swept trace, zero velocity and stop algorithm,
3. If $f_{n+1} = 1$ and player would be stuck at $\mathbf{r}_{n+1}$ according to unswept trace, zero velocity and stop algorithm,
4. Move player to $\mathbf{r}_{n+1}$,
5. Decrease time left to $t_{n+1} = t_n - f_{n+1}t_n$,
6. If $t_{n+1} = 0$ (or equivalently $f_{n+1} = 1$), stop algorithm,
7. Update velocity with surface normal $\hat{\mathbf{n}}_{n+1}$ to $\mathbf{v}_{n+1} = \mathbf{v}_n - (\mathbf{v}_n \cdot \hat{\mathbf{n}}_{n+1})\hat{\mathbf{n}}_{n+1}$,
8. If amount of surfaces collided with is four ($n + 1 = 4$), stop algorithm, otherwise increment n by one and return to step 1.

It had not been mentioned, but the game only allows up to four collisions before giving up as uneven terrain can lead to a lot of collisions. There are also swept and unswept stuck tests which removes all of the player's speed and stops further movement for that tick if the player is or would end up in a peculiar spot. Another unmentioned special case is when a player gets wedged into a crease, i.e. two surfaces forming a v-shape. If that happens the game takes the cross product of the two surface normals and sends the player along the resulting direction. Forgetting to account for this would lead to a kind of oscillation between both surfaces without ever fully removing the player's parallel velocity.

It is important to note that the stuck tests are only used to prevent the player from getting into a stuck position. If the player is actually stuck, then step 1 from Section 2.2.2 will kick in after an initial delay of at most one second, where a continuous search for a place to free the player takes place. This is done by systematically choosing one of many preset offsets around every 0.05 seconds. If the chosen offset during a check does not work, then the game will skip most of the player tick and try again next time. It is not guaranteed that the player will ever get unstuck as the number of planned offsets is limited.

8.1 Edgebug

Sometimes the player can hit a spot very near the edge of a platform and somehow keep all of their horizontal velocity while also completely stopping their fall after the collision. This is called an edgebug and can be used to negate fall damage or prevent the player from failing a jump by edgebugging through a teleport trigger. It works of the fact that the player can hit the ground and then still go a bit forward

all within the same tick as described above. This allows the player to land near the edge of a platform, collide with it, and simply go directly off it before any grounding check occurs. This means the player starts and ends the tick airborne and therefore receive no speed penalty while the collision zeros out their vertical velocity. The reason it allows the player to totally ignore a trigger is because triggers only gets activated sometime after movement has been done, which can be seen in the Section 2.2.2.

Another way to apply the idea behind edgebugs is to make use multiple surfaces collisions within a single tick. This makes it possible to perform so-called slidebugs where the player first hits the ground as with an edgebug, but they do so right in front of a ramp instead of an edge. Then if there is enough time left for movement, the player can hit the ramp, possibly giving them a vertical velocity above 250 units per second such that they cannot get grounded. This method preserves more speed than simply falling directly onto the ramp as the collision is comparatively less direct.

Because edgebugs are so good at preserving speed while also getting rid of all vertical momentum a lot of jump mappers have incorporated so-called auto-edgebugs into their maps. Their construction is very simple as it is just a slope the player slides down with a very short flat floor sticking out at the very bottom. This way the player can build up speed while sliding down the ramp and edgebug at the end. Normally the player also shoots rockets while on the ramp and/or right after the edgebug. Auto-edgebugs have a well-known problem though. Sometimes the player just stops at the very bottom instead of edgebugging like they are supposed to. This is because the player can get unlucky and run out of time for movement on the tick where they hit the short edgebug platform. This results in the player being right above the platform, meaning they get grounded right after and lose most of their speed on the next tick where the walking speed is enforced.

9 Collision

It is well-known that world collision in Source games can be quite buggy at times, but the swept and unswept traces that are used for this are still somewhat of a black box. So to explain the bugs it is necessary to get a good idea of how traces actually work. Unswept traces are generally not too interesting because their only function is to check if the player is stuck at a given location. Swept traces, on the other hand, are a lot more interesting as they can figure out when and where the player will collide with a surface given a start and an end position. They also have some quirks that gives rise to most of the collision bugs.

One of these quirks can easily be spotted by enabling `cl_showpos` and walking straight into an axis-aligned wall. By doing this and looking at the displayed coordinates, it becomes clear there still is a tiny gap between the player and the wall. Though the displayed distance is 0.03 units, it is in reality 0.03125 units that has been rounded down. The separation exists because older Source games limited the numeric precision of player coordinates sent from the server to the client. This meant that if the player was too close to the wall on the server, the client could end up receiving a position that would place them inside the wall. This precision problem has been amended in TF2, but the imposed gap still serves the purpose of preventing the player from getting too close to walls where all the collision bugs start appearing.

Luckily for players that want to wallbug, the apparent 0.03125 extension of any surface is not impossible to get through. This is because the extension is only applied when the player would collide with the non-extended surface. Therefore if the end position of the swept trace would place the player within 0.03125 units of the surface, no collision would occur and the player could be moved there. In practice this is very easy to do as the player simply needs to stay up against any wall and wiggle from side to side in a steady tempo. From here, all of the collision bugs that will be covered becomes available and can be done under the right circumstances.

The effectiveness of wiggles is a result of wall-hugging in the air, which was touched on in Section 7.1.1. In this case, the parallel speed is not the interesting part, but instead the speed going into the wall. This speed will continue to decrease while wall-hugging since the acceleration each tick is proportional to $v_{\text{cap}} - \mathbf{v} \cdot \hat{\mathbf{v}}_{\text{wish}}$. That way the perpendicular speed can become arbitrarily small and any tiny distance to the wall can be traversed without hitting it. Getting as close as the game allows to any wall is therefore simply a matter of wiggling from side to side until the player gets a perpendicular speed that makes the player move right up against the wall. The optimal wiggle interval is only around 0.15 to 0.2 seconds because very low speeds are unable to move the player at all as a consequence of precision issues.

9.1 Traces

To figure out how all the bugs surrounding brush collisions can exist, it is essential to know more about the inner workings of traces. As mentioned in Section 8, traces are either unswept and swept. An unswept trace simply checks if the player would be stuck at some position $\mathbf{r}$. Swept traces are used for collision detection, so they need a start $\mathbf{r}_1$ where the player currently is and some endpoint $\mathbf{r}_2$ where the player is trying to go. A swept trace would then return the position of the collision $\mathbf{r}_{\text{hit}}$. It also calculates the fraction traveled $f = |\mathbf{r}_{\text{hit}} - \mathbf{r}_1| / |\mathbf{r}_2 - \mathbf{r}_1|$ that tells how far the player got before getting stopped. To know what the player hit, the surface plane is also returned by the trace, containing its normal $\hat{\mathbf{n}}$ and its signed distance to the origin d .

Moving on to the implementation of traces, the problem of knowing which brushes to check comes up, since going through all brushes in the map would be very inefficient. To deal with this, the game has something called nodes and leaves, which are stored directly in the BSP file of the map. Each node splits the whole world in two with a plane, where on each side is another node. This way it is possible to quickly navigate to a node near the player by constantly choosing the next node to be the one that is on the same side of the plane as the player. At the end of this node search there is a leaf which contains information of what is inside it, most notably brushes. It is possible to visualize the leaf the player is currently in by using `mat_leafvis 1` or all leaves at the same time with `mat_leafvis 3`. If the trace start and end position are in different leaves, then the brushes in all the leaves in between will also be checked. There are usually node planes placed along each visible brush side, so a trace often visits multiple leaves under a brush collision.

The actual brush collision checks also make use of planes in a similar way to nodes by converting all brush sides into planes. While planes do have two sides, they also have a normal that can be used to give them an explicit direction. Therefore all the brush planes can be made to point out of the brush and make it possible to determine if a point is inside it or not. More precisely, if the player is on the outside of any plane of the brush, then they are not touching it, meaning they have to be inside all of the planes to be considered stuck.

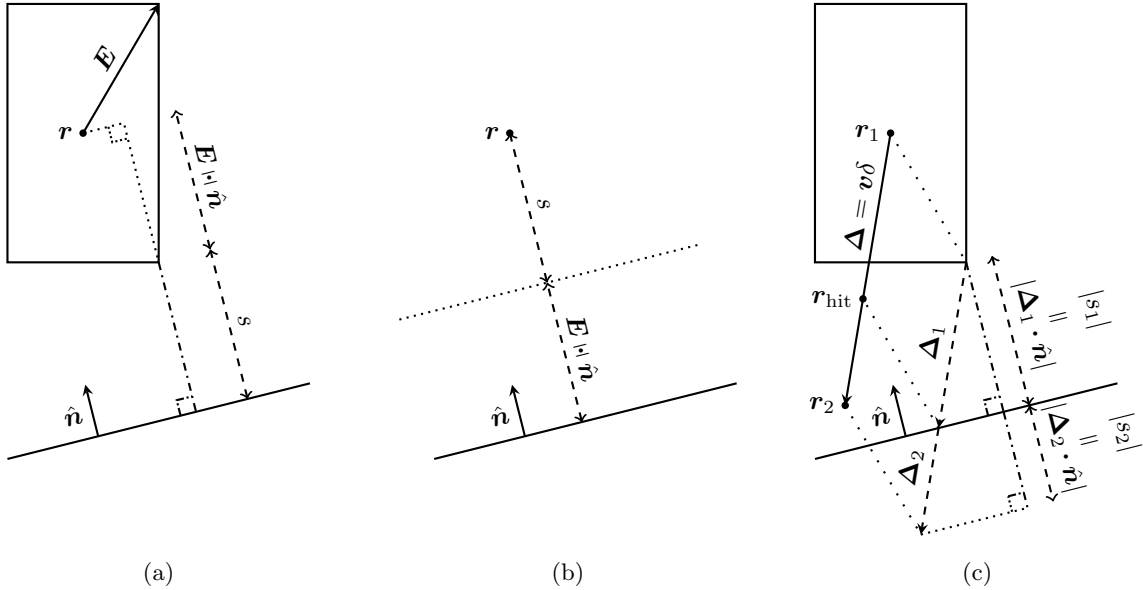


Figure 3

To understand how all of this is done mathematically, it is best to first focus on only unswept traces, as they are simpler and a lot of points that will be made are used for swept traces too. To start with, instead of using the player origin, the game opts to use the center of their bounding box $\mathbf{r}$ for traces. This allows for the useful definition of the extents $\mathbf{E}$, which is the offset from $\mathbf{r}$ to the corner of the bounding box such that all components of $\mathbf{E}$ are positive. This way, only $\mathbf{r}$ and $\mathbf{E}$ are needed to know the player's position and bounding box.

The method for figuring out if the player is outside a brush side is done by calculating signed distance to the accompanying plane. For a point the signed distance to the plane would simply be $s_0 = \mathbf{r} \cdot \hat{\mathbf{n}} - d$, where $\hat{\mathbf{n}}$ and d are the plane normal and signed origin distance respectively. However, this is not taking

the bounding box into account because $\mathbf{E}$ and $\hat{\mathbf{n}}$ do not work well together. The extents were defined to be all positive, but the plane normal can point in any direction, which makes it necessary to somehow handle their differing signs. To better understand how to get around this, assume $\hat{\mathbf{n}}$ is pointing in a direction opposite to $\mathbf{E}$. That way the closest point of the bounding box to the plane would be the corner $\mathbf{r} + \mathbf{E}$, giving the distance $s = (\mathbf{r} + \mathbf{E}) \cdot \hat{\mathbf{n}} - d = \mathbf{r} \cdot \hat{\mathbf{n}} + (\mathbf{E} \cdot \hat{\mathbf{n}}) - d = s_0 + \mathbf{E} \cdot \hat{\mathbf{n}}$. Because $\hat{\mathbf{n}}$ was chosen to be opposite of $\mathbf{E}$, the dot product can also be written as

$$\mathbf{E} \cdot \hat{\mathbf{n}} = -(|E_x \hat{n}_x| + |E_y \hat{n}_y| + |E_z \hat{n}_z|) = -(\mathbf{E} \bowtie \hat{\mathbf{n}}), \quad (4)$$

where $\bowtie$ is a made-up operator that can be thought of as an absolute dot product. The final rewrite of the distance then becomes $s = \mathbf{r} \cdot \hat{\mathbf{n}} - (d + \mathbf{E} \bowtie \hat{\mathbf{n}})$. The nice thing about this expression is that it works for any $\hat{\mathbf{n}}$ because the absolute values in $\mathbf{E} \bowtie \hat{\mathbf{n}}$ get rid of any sign dependency (see Figure 3a). The operation can be seen as mirroring the whole world around the player in such a way that $\hat{\mathbf{n}}$ becomes opposite of $\mathbf{E}$. Another take that is easier to understand is that the player becomes a single point by losing their bounding box ($\mathbf{r} + \mathbf{E} \rightarrow \mathbf{r}$), while the plane gets extended outwards to match the disappearance of it ($d \rightarrow d + \mathbf{E} \bowtie \hat{\mathbf{n}}$). This is illustrated in Figure 3b where $\mathbf{E} \bowtie \hat{\mathbf{n}}$ changes its anchor point from $\mathbf{r}$ to the plane.

With this formula unswept traces just check if $s > 0$ for any of the brush planes to conclude that the player is outside of the brush. Logically, this also means if $s \leq 0$ for all planes, then the player should be considered stuck inside the brush.

Swept traces are a bit more complicated as they have both a start and end position, symbolised by $\mathbf{r}_1$ and $\mathbf{r}_2$ respectively. With these comes the traveled vector $\Delta = \mathbf{r}_2 - \mathbf{r}_1$, which does not necessarily point straight into the brush side that the player will collide with. The distance the player travels before hitting the brush is therefore longer than their distance to the plane s that was used for performing the unswept trace. Luckily the plane distance method can be used for swept traces too, but its place in the calculations needs to be derived first (use Figure 3c to make it easier to follow along).

The first thing to do is to move the base of Δ from $\mathbf{r}_1$ to the corner of the bounding box that will collide with the brush side. From here Δ can be split into two: Δ_1 that goes from the player's corner to the point of collision, and Δ_2 that continues from the end of Δ_1 to the end of Δ . With these in place it is possible to express the fraction of the total distance the player moved before the collision with $f = |\Delta_1|/|\Delta|$, such that the point of collision will be $\mathbf{r}_{\text{hit}} = \mathbf{r}_1 + f\Delta = \mathbf{r}_1 + f(\mathbf{r}_2 - \mathbf{r}_1)$. The goal is therefore to find a nice expression for f , as everything else can be calculated using that.

This task ends up being quite simple after realising the projection $\Delta_1 \cdot \hat{\mathbf{n}}$ is equal to the player's distance to the plane when their position is $\mathbf{r}_1$, with a possible sign difference. The same argument also works for $\Delta_2 \cdot \hat{\mathbf{n}}$ when the player is on the other side of the plane at $\mathbf{r}_2$. The plane distances can be calculated using the same method that was used for the unswept trace and will similarly be written as s_1 and s_2 . Calculating the fraction is now a simple task when being careful with the signs

$$f = \frac{|\Delta_1|}{|\Delta|} = \frac{|\Delta_1 \cdot \hat{\mathbf{n}}|}{|(\Delta_1 + \Delta_2) \cdot \hat{\mathbf{n}}|} = \frac{|s_1|}{|s_1 - s_2|}. \quad (5)$$

This seems like a proper result, but it is not quite how the game does it. That is because it is still missing ε somewhere to try and prevent the player from moving too close to the brush. The game implements this by tweaking the fraction into

$$f = \max\left(0, \frac{s_1 - \varepsilon}{s_1 - s_2}\right), \quad (6)$$

such that the fraction will be shortened a bit, making the player stop just a bit short of the brush as if the plane was moved ε forward. The extension is assigned the value $\varepsilon = 0.03125$ and it is the exact same surface extension that was discussed in Section 9. The max function is there to prevent the fraction from becoming negative, which would have resulted in the player moving backwards. For a collision, it is a given the player is starting outside and tries to end inside the brush, meaning $s_1 > 0$ and $s_2 \leq 0$, making it possible to not deal with absolute values. However, those are not the only signs s_1 and s_2 can take, so a swept trace can as such be split into four different scenarios:

1. $s_1 > 0, s_2 > 0$: There will be no contact with the brush along the trace,
2. $s_1 > 0, s_2 \leq 0$: Player starts outside, but will collide with the brush at some point,
3. $s_1 \leq 0, s_2 \leq 0$: Player is fully stuck inside the brush and will not be able to move,
4. $s_1 \leq 0, s_2 > 0$: Player starts stuck, but is able to get out by going to the end position.

Case 2 was just discussed in detail. For the rest, the simplest is case 1 which just moves the player to $\mathbf{r}_2$ with $f = 1$. It does come with the consequence that the player can end up closer than ε to the brush if $\mathbf{r}_2$ happens to be within that interval. This is what makes it possible to wiggle past the surface extension. Cases 3 and 4 are similar in that the player starts inside a brush, but for the latter the player is able to escape the brush. If that happens the player just moves to $\mathbf{r}_2$ like in case 1, but the trace also returns a fraction conveying how far the player had to move to leave the brush.

9.2 Wallbug/Rampbug

There are times when the player will suddenly lose all their speed when nudged up against a surface, most notoriously wallbugs and rampbugs. Wallbugs are generally referred to as getting stuck on a fully vertical wall, but it can be used with some leeway. Rampbugs can happen while the player is sliding/surfing along a ramp and results in the player making an abrupt stop. It should be mentioned that these rampbugs are not the same kind of rampbug that were discussed in Section 5.1. While they both rob the player of their speed on a ramp, they occur for very different reasons. The rampbug that is the focus of this section happens because the player gets too close to some geometry, while the other rampbug relies on the player getting grounded, putting them back down to walking speed.

There can be various kinds of geometry in a map, each using one of several ways to handle collisions. Examples of these are brushes, displacements, and physics props (VPhysics). When traces were described, it was only mentioned how they work for brushes. This is because the collisions against displacements and physics props are a lot more complicated and are therefore also a lot more difficult to figure out/describe. As a consequence, only bugs related to brushes will be explained, but luckily(?) there is no lack of those.

Most wallbugs that are encountered are on walls that are not aligned with the world axes. These are commonly called angled wallbugs and work for the same reasons as rampbugs. Because of this we can focus solely on angled wallbugs with all the results also being applicable to ramps. The main upside to this is that angled wallbugs and rampbugs are actually two different, but closely related, bugs and walls just happen to make their distinction clearer. By wiggling up against an angled wall at an appropriate tempo, it becomes apparent that the player can either get stuck for a single moment or stay stuck indefinitely by pressing up against the wall. These are the two different types of angled wallbugs and are respectively caused by the unswept and swept stuck checks from Section 8.

Both the swept and unswept traces have two steps they follow. First they find what leaves that should be visited and then what brush the player will collide with or is/would be stuck in. It was mentioned earlier that almost every visible brush side is accompanied by a node plane that splits each side into leaves, one without brushes and one containing the brush in question. As such, whenever the player should interact with a wall, multiple calculations of $s = \mathbf{r} \cdot \hat{\mathbf{n}} - (d + \mathbf{E} \cdot \hat{\mathbf{n}}) \leq 0$ are done. For swept traces, both the start and end position are checked for being inside the leaf containing the brush, and then the same check is done again against the brush itself. Unswept traces go through the same, but only have a single position that needs to be checked. They do have one minor difference in that the player's bounding box is figuratively extended by one unit in each direction when searching for leaves to be extra thorough. This means unswept leaf finding can be disregarded as players need to be a lot closer to a wall to wallbug.

It is in these checks that angled wallbugs appear. The cause comes from the checks not agreeing on what side of the mutual plane the player is on. While all of the relevant checks do use $\mathbf{r} \cdot \hat{\mathbf{n}} - (d + \mathbf{E} \cdot \hat{\mathbf{n}}) \leq 0$ or mathematically equivalent versions thereof, it is not guaranteed they will give the same answer when done on a computer with limited precision. To get a better idea of what goes wrong, the calculations that a server running on Linux goes through are

$$\begin{aligned} \text{swept leaf check} : n_x r_x - d + n_y r_y + n_z r_z &\leq |n_y E_y| + |n_x E_x| + |n_z E_z|, \\ \text{swept brush check} : (n_y r_y + n_x r_x + n_z r_z) - (|n_x E_x| + |n_y E_y| + d + |n_z E_z|) &\leq 0, \\ \text{unswept brush check} : n_x (r_x + (\mp E_x)) - d + n_y (r_y + (\mp E_y)) + n_z (r_z + (\mp E_z)) &\leq 0, \end{aligned}$$

where all operations are done in the order they are written. With some rearranging it is found that they are all equivalent to $\mathbf{r} \cdot \hat{\mathbf{n}} - (d + \mathbf{E} \cdot \hat{\mathbf{n}}) \leq 0$, but the actual takeaway is the arithmetic is not done in the same order for any of the checks. This can lead to tiny errors that might end up changing one or more checks from true to false or vice versa.

The angled wallbug where the player only stops for a moment is the easiest to explain. It is caused by the swept trace routine (leaf or brush check) not finding anything at $\mathbf{r}_{\text{end}}$. This means the player should be moved to $\mathbf{r}_{\text{end}}$, but according to step 3 in the movement algorithm of Section 8, the unswept check

also needs to agree. As just discussed, there is a chance it thinks otherwise, so the player will suddenly lose all their speed. However on the next tick the velocity change will have voided this conflict and the player will start to fall once again.

The other angled wallbug lets the player stay on a wall by simply pressing against it. The way it happens is if only the swept brush check returns true at $\mathbf{r}_{\text{end}}$. This leads to the player moving to $\mathbf{r}_{\text{end}}$ because the swept brush check never got reached which would otherwise have registered a collision with the wall. On the next tick $\mathbf{r}_{\text{end}}$ has now become $\mathbf{r}_{\text{start}}$, while the new $\mathbf{r}_{\text{end}}$ is assured to be inside the wall as long as the player is pressing against it. Now the swept leaf check will return true and lets the swept brush check get executed. However this check considers $\mathbf{r}_{\text{start}}$ to be inside the wall and consequently takes away the player's speed by following step 2 in the movement algorithm. This will continue to happen indefinitely until the player cancels the wallbug themselves. When they do, $\mathbf{r}_{\text{start}}$ and $\mathbf{r}_{\text{end}}$ will have the same horizontal position, meaning both the swept leaf check and unswept brush check return false, allowing the player to fall.

9.2.1 Coordinate wallbug

Angled wallbugs rely on there always being some position along the wall where the checks disagree, but that method does not apply to axis-aligned walls where any parallel movement give the same trace result. However, it is known that it is possible to wallbug on axis-aligned walls that are placed around powers of two along an axis. These are appropriately called coordinate wallbugs, but their explanation is a bit more complex as an understanding of how real numbers are represented in computers is needed.

Any real number can be approximated using a decimal number like 123.456, but it could also be written in the scientific notation $+1.23456 \times 10^2$ that contains the sign +, the significant 1.23456, the base 10 and the exponent 2. Computers also use this method to represent real numbers, but they of course do it in binary. The base that is used is therefore base 2, which is fixed, such that only the other three parts need to be physically stored. The game uses 32 bits for these: 1 bit for the sign, 8 bits for the exponent, and 23 bits for the significant. The precision of a number is therefore restricted by the amount of bits that are available, but also by how large the exponent is. The last point is important to understanding coordinate wallbugs, so it is a good idea to have examples to look at. So going back to 123.456, imagine the significant can only have six digits, giving $+1.23456 \times 10^2$ as before, which is accurate. However another number like 12345.678 has more digits than this and will have to be rounded to up to 12345.7 so it can fit into its limited scientific form $+1.23457 \times 10^4$. Larger numbers therefore have less precision because only the most significant digits are stored. The same idea is true for the 32-bit binary numbers, but their examples would just be pretty big.

The limited precision can become a problem when adding or subtracting numbers as some rounding can end up happening. This is exactly what coordinate bugs exploit and it is easiest to see how they work by using an example. If a wall is placed 524 units away from the origin along the x-axis, then the player can use the wiggle technique to get as close to the wall as the 32-bit numbers allow, meaning the position the swept trace would try to move them to would be just below 500 when taking the 24 wide extent into account (48 total width). Since the swept trace tries to move the player while they are this close to the wall, it means it did not detect a collision. It therefore returns a fraction of one, making the game employ an unswept stuck check at the end position. For this example, the condition is $-(r_x + E_x) - d \leq 0$ where $d = -524$, $E_x = 24$, and r_x is just below 500. The important part happens when $r_x + E_x$ is calculated and doing the addition by hand is the most straightforward way of seeing it:

1. Write r_x and E_x out in their floating point form

$$r_x = +1.111100111111111111111111 \times 2^8 = 499.999969482421875$$

$$E_x = +1.1000000000000000000000 \times 2^4 = 24$$

2. Convert to normal binary notation without exponents and pad with some zeros

$$r_x = 0111110011.11111111111111110000$$

$$E_x = 0000011000.00000000000000000000$$

3. Add them

$$r_x + E_x = 1000001011.11111111111111110000$$

3. Convert result to floating point form and remove trailing zeros

$$r_x + E_x = +1.000001011111111111111111 \times 2^9 = 523.999969482421875$$

The result is 523.9999... as expected, but it currently uses 24 digits after the ‘1.’, requiring the result to be rounded so its significant can fit into the limited 23 bits. The default method is to round to the nearest value, but in this case there is only one binary digit to round, so both rounding directions are equally close. The tie-breaker rule for this kind of case is to round the number so the last digit of the rounded number is even, which in binary means the last digit has to be zero. Therefore the result is rounded up, yielding $r_x + E_x \approx +1.0000011\bar{0} \times 2^9$, which is exactly 524 in decimal. The unswept check has now become $-524 - (-524) \leq 0$, resulting in the player being considered stuck at this position. Just like the unswept stuck check on an angled wall, the player will only be stopped for this tick before they begin falling anew, though it is usually only for a brief moment before the coordinate wallbug happens again.

This example was not something that was hard to find, in fact, it was very easy to do so. The reason the result was rounded up is because adding E_x to r_x made it enter a higher power of two, going from 2^8 to 2^9 . Using that train of thought, the wall could have been placed anywhere within the half-open interval $(2^9, 2^9 + E_x]$ and still have let the player perform a coordinate wallbug. Generalizing this for even more powers of two and all cardinal directions, coordinate bugs occur when walls are placed within $\pm(2^n, 2^n + E]$ for $n \geq 4$. The condition on n is there because the player’s position will not sum to a higher power when the wall is placed less than $2^4 = 16$ from the origin of the map when $E = 24$.

While the condition seems to imply that coordinate bugs will disappear when close to the origin, it actually gives rise to another variant of coordinate bugs with an even stronger effect. It does come with the caveat that the brush must be in the shape of a box, but that should not be a pose a problem to mappers. This extra requirement is there because the geometry of box brushes is so simple that they have their own optimized routine for computing traces. As a result, some traces can give different results depending on what type of brush the player is colliding with, though these differences generally only appear when sought out.

The unswept check is actually functionally the same for both types of brushes, so nothing special there. The same story is not true for the swept check. For non-box brushes, the swept check can never induce a coordinate wallbug because the signed plane distance and extent are added together first (e.g. $r_x - (E_x + d) \leq 0$), meaning the player’s position will not enter a higher power. Contrarily, swept traces against box brushes use the last possible pairing, that is, calculating the difference in position between the player and the wall as the first thing. That means the trace checks the player’s start position using a condition that is effectively $0 < |w - r_x| - E_x$, where w is the coordinate of the wall along the axis it is perpendicular to. The roles of the extent and the wall have now been reversed as it will be w that changes r_x . The same kind of calculations as before can be done and it turns out the wall can be placed within $\pm(8, 40]$ to make this type of coordinate wallbug happen (remember the brush has to be a box).

It was mentioned the effect of this coordinate wallbug would be stronger, but not in what way. The previous type used the unswept trace to stop the player by achieving the smallest possible separation, but in this case, the unswept trace will no longer trigger. The player can therefore close the distance, from where the coordinate wallbug will continue to happen every tick since there is nothing to move them away from the wall. The result is the player hovering in the air while against the wall without even needing to press anything. The swept trace does consider the player to be stuck, but since step 1 in Section 2.2.2 uses an unswept trace, nothing is done about it. Luckily the player is able to escape by themselves by moving away from the wall because swept traces also contain functionality for exiting a brush.

9.3 Pixelwalk

Knowing how the player collides with a surface is all well and good, but the thing about brushes is that they have multiple sides. After all, there might be more than one sides that fall under scenario 2 from Section 9.1. It might not be very obvious how this could happen, so try imagining a triangular bump with the player falling fast from above while fully on the right side of it (or use Figure 4a). It is clear the right side falls into case 2 when the player is going to land, but the same is also true for the left side. That is because it is internally a plane and therefore extends infinitely to the right, ending up right under the player. Now, because of the brush sides meet at an angle, the right side of the left plane will always end up being above the right side of the right plane. That means it will always take longer to reach the right plane when the player is located on the right side. A similar thing happens when the player is on the left, where the left side requires more travel time. So the way to choose the correct brush side boils down to finding the brush plane that the player would collide with last. This can also be expressed as choosing the side with the highest trace fraction f , which is exactly what the game does.

That is not quite the end though, as the situation gets a bit more complicated if the player placed above the center of the triangle. A collision with either of the two sides would produce an unexpected result where the player hits the plane-only parts of the brush sides, i.e. thin air. The game solves this by having an invisible horizontal plane at the top of the bump, which is there solely for this type of collision (Figure 4b). The player will end up hitting this plane instead of the other two using the same selection process, giving a collision appropriate for the axis-aligned bounding box the player has. This fix can be generalized for any brush that is missing axis-aligned planes that are necessary for a proper collision model. In fact, it is as easy as figuratively enclosing the brush in the smallest possible box and adding the sides that it does not already have as invisible planes.

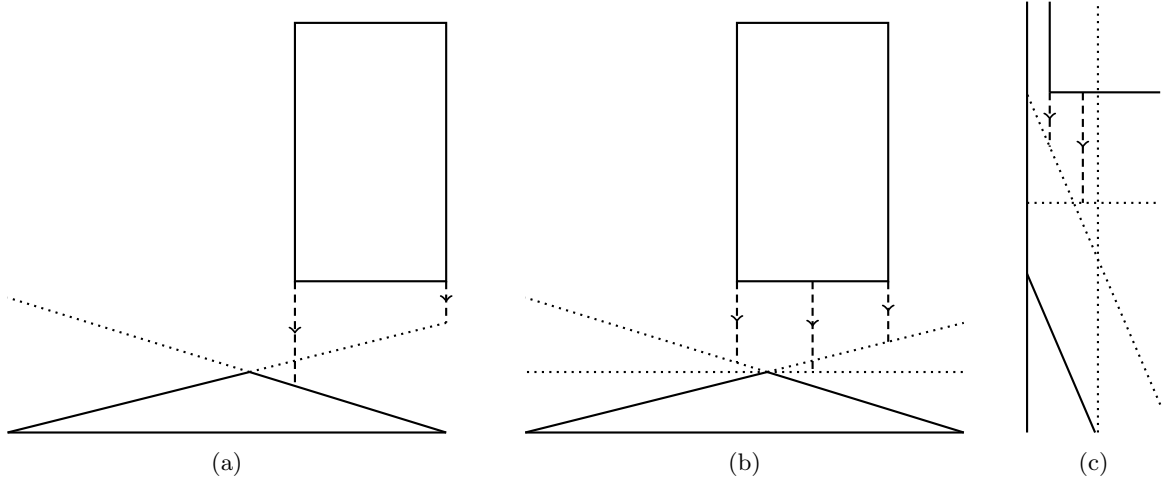


Figure 4

These methods work great for producing brush collisions, but they contain a hidden flaw. The approach of choosing the sides based on the player's travel time is perfectly sane, however using the trace fractions for this is a problem. That is because the fractions have the offset ε in their numerator, so they do not correspond to the true travel time to the brush sides. One of the consequences of this is pixelwalking. They allow the player to walk on top of a triangular brush that is flush with a wall. They are even easy to execute, just apply the old trusty wiggle technique to get super close to the wall and a pixelwalk will naturally happen. Since the player is able to magically stand somewhere without any standable surface, it is obvious that the invisible horizontal plane is at play somehow. It all comes down to the sloped and the horizontal plane effectively getting extended by ε . So to understand pixelwalks, imagine the extended planes and the player falling directly towards them (or look at Figure 4c). The horizontal plane has simply been raised up by ε everywhere, but for the sloped plane it depends. Far away from the wall, the sloped plane is under the horizontal plane, but at some point when nearing the wall, this relationship switches. By being close enough to the wall, it is therefore possible to make the player collide with the horizontal plane instead of the intended sloped plane.

The player is also able to stay on the horizontal plane because the game constantly checks 2 units below for standable ground using swept traces, making the bug repeat every tick. That fact does hinge on one more important detail though. When the player collides with the horizontal plane, they get placed right at the ε extension, meaning any following swept traces will compute a fraction of zero for both planes, resulting in a stalemate. In that case, the plane that was processed first becomes the chosen plane. It just so happens the axis-aligned planes always come before any sloped planes, such that the horizontal plane is always chosen over the sloped plane. As a consequence, the player can stay floating as long as they want.

9.4 Flush brushes

The next logical step is to figure out what happens if multiple brushes are in the way of a swept trace. The most basic case is when the player starts a bit distanced from the brushes, because then the brush that would be hit first is chosen, which is of course done by comparing the fractions. While the idea is sound for normal gameplay, it will fall apart if the player is within ε of more than one brush. Those brushes will all return a fraction of zero, producing yet another stalemate of trace fractions. Similarly

to brush sides, it is the brush that is tested first which will be used in the trace result. There are two factors that decide the brush checking order: the order in which leaves are visited and the index of the brush used internally within a leaf.

Because the game checks leaves one at a time, the leaf order ends up being the biggest deciding factor for this problem. It is decided by how the nodes and leaves were split using planes, where the node/leaf on the positive side of the splitting plane gets visited first. For splittings that point along an axis, the plane simply gets pointed in the positive direction of that axis. This makes the order somewhat predictable as it will almost always be the leaf furthest along an axis that will get visited first. The obvious exceptions would be if two leaves are further along different axes or if the splitting plane is angled. However both of these scenarios are not that interesting in terms of discovering new bugs, so the focus will be on leaves or brushes that are split along an axis. On the off chance that finding the ordering of leaves for the two exceptions becomes necessary, it can be done by simply enabling `mat_leafvis 1` and `developer 1`. The first command will draw the leaf that the player's viewpoint is inside, while the second command will make the leaf index output to the console. The leaf index corresponds to where it is stored in the BSP leaf array and it just so happens that leaves with lower indices will always get visited first.

The second factor, the brush index for a leaf, is a lot simpler. Every leaf has a list of brushes that are inside it. Every time a leaf is visited, it will go through this list in order to look for collisions. The order of the list can very easily be controlled by the mapper as it the opposite of the brush listing in the VMF file. That means brushes that are placed later will be prioritized over earlier brushes. A Standard brush will always split the node/leaf tree at its sides, so most of the time a leaf will only contain a single brush. However the mapper can choose to use a `func_detail` entity which do not induce any splitting. Their intended use is for optimizing the rendering of brushes as the nodes and leaves are also utilized for that, but it ends up having the unintended side effect of slightly altering how collision is done.

With that out of the way, it is now time to discuss the collision bugs that can occur along brushes that are flush with one another. While it is possible to bug flush brushes while going horizontally along them, the exploits of interest happen because of falling. Also to avoid wallbugs, the flush brush sides should also be axis-aligned. So all in all, the focus will be on two brushes stacked vertically with their flush sides being axis-aligned. From this, the goal is to somehow collide with the top side of one of the brushes which the player visually does not have access to.

There are a few more requirements that can be squeezed out from this setup. As the player will be against the wall while falling, their fraction to the flush sides will be zero. As a consequence, the top side will also need a fraction of zero for a bug to occur. That is because a top side with a non-zero fraction will be used as the result for the brush it is part of, but the other brush will be chosen in the end because it is the closest to the player with its fraction of zero. This ends up automatically filtering out non-box brushes because sides that point in the x and y directions get processed before the z-pointing and slanted sides. Therefore if both the top side and the wall side of a brush must have a fraction of zero for a bug to happen, it becomes impossible for the top side to ever get chosen. That means only box brushes are left a possibility.

A box brush handles collisions a bit differently compared to a non-box brush. Its main use is to optimize the process of calculating the fractions and choosing the correct side by using a method (SIMD instructions) that lets it evaluate all sides in parallel. The fractions are calculated using an equivalent formula to Equation (6). The real difference lies in how the side that should be collided with is selected. That task is split into two steps: finding the relevant axes and comparing the front facing sides along these axes. For calculating fractions, only cases 2 and 4 of Section 9.1 need to be considered. So the relevant axes are those the player is trying to cross with the swept trace. As a reference, if the player is pressing directly against a big wall, then they will only be crossing the axis they are trying to go along, while being fully inside along the other two axes. Each axis has two sides, but only the one that is facing the opposite direction of where the player is going can be collided with, meaning these are the ones to be checked. With the relevant axes and sides found, it is now time to figure out which side should be used for the trace result. It follows almost the same logic that is used for non-box brushes where the greatest fraction gets chosen. The only difference lies at what point in the fraction calculation that they are compared. Non-box brushes compare Equation (6), but box brushes decide to compare them before the max function is applied. So to bug on the top side of a box brush, the following inequality needs to be satisfied

$$f'_{\perp} = \frac{s_{\perp} - \varepsilon}{|\Delta_{\perp}|} < \frac{s_z - \varepsilon}{|\Delta_z|} = f'_z, \quad (7)$$

where the subscript $\perp$ (perpendicular symbol) is used to symbolize one of the wall sides as they are

perpendicular to the z-axis. The denominators come from the fact $\Delta_i = -\Delta \cdot \hat{n}_i$ where $\hat{n}_i$ is the normal of a side opposing the player's travel direction.

9.4.1 Pixelsurf

A natural extreme of pixelwalking would be to somehow stand on the invisible horizontal plane without the visible side being angled at all. Sadly, this is impossible to do because the 2 unit grounding trace will not be able to find a brush directly below the player. However that does not exclude the possibility of colliding with the horizontal plane. Since this is now a problem concerning two flush brushes, only a box brush will be viable, at least for the bottom brush. The plan is to collide with the top side of the bottom brush by having the player fall within 0.03125 of the top side while pressing up against the wall. As gravity is applied each tick, it should be possible to collide with the top side every tick, letting the player stay suspended mid-air while being also being able to slide horizontally along the wall, resulting in a so-called pixelsurf.

While it sounds cool in idea, making it work requires more thought. The first problem it faces is the brush checking order. If the two flush brushes are made from normal brushes, then they will be split into two different leaves. However, because the bottom brush is necessarily a box brush, it means the plane splitting the two leaves will be pointing directly up along the z-axis. That means the top brush would always be checked first and therefore chosen over the bottom brush. As a result, pixelsurfs can only work if both brushes are within the same leaf and the bottom brush is listed first in the leaf. As previously discussed, `func_detail` allows for this to happen in a controlled manner, but because it is not used that often for walls in jump maps, it ends up making pixelsurfs a rarely seen thing and that is without even taking the 0.03125 distance requirement into account.

That leads directly into the next problem: the pixelsurf interval is stricter than 0.03125. This interval can be derived from Inequality (7) by rearranging it and plugging numbers in afterwards. Before doing that, it is best to first look at what the ideal value of $s_\perp$ would be. As it contributes to the left side that should be as low as possible, the best possible action would be to make $s_\perp = 0$. While it technically is not possible to make it zero, wiggling against the wall makes it come so close that it might as well be. So with that assumption in mind, the inequality becomes

$$s_z > \varepsilon(1 - |\Delta_z|/|\Delta_\perp|), \quad (8)$$

which is pretty nice looking. The last part is now to figure out the values of the terms on the right-hand side. The $|\Delta_z|$ is just $(g\delta)\delta = (800 \cdot 0.015) \cdot 0.015 = 0.18$, as seen from Section 2.2.2 with half of the gravity coming from the previous tick and half from the current tick. Likewise, $|\Delta_\perp|$ is also possible to evaluate by referring to Section 7 and looking up the necessary values in Table 1. Plugging into Equation (3) then gives

$$|\Delta_\perp| = |v_\perp|\delta = \min(240 \cdot 10 \cdot 0.015, \max(0, 30 - 0)) \cdot 0.015 = \min(36, 30) \cdot 0.015 = 0.45. \quad (9)$$

This finally results in $s_z > 0.03125 \cdot (1 - 0.18/0.45) = 0.01875$. As s_z should also be less than ε , the player needs to be within (0.01875, 0.03125) units above the brush to do a pixelsurf, a measly 0.0125 unit window. This, together with the brush collision order, is why pixelsurfs are pretty much never seen in TF2. They are a lot easier to find in CS:GO because the leaf visiting is inverted, such that the bottom brush usually gets checked first. In addition to that, crouching also lowers the air acceleration down to 34%, which in turn makes the pixelsurf window bigger.

9.4.2 Texturebug

The biggest obstacle for finding bugs that stop the player from falling when against a pair of flush brushes is that the top brush will always be checked first whenever only default brushes are placed. So unless the mapper made use of some special brushes, it is simply impossible to land on the top side of the bottom brush like pixelsurfs do. The saving grace is that all that matters is that the top side is chosen in Inequality (7), so the player does not necessarily need to physically hit a top side to register a collision with it. Since landing on the bottom brush was determined to be impossible, this hits towards the possibility of the player somehow colliding with the top side of the top brush while being directly next to it. As odd as it sounds, this is actually possible because of how the relevant axes and sides are decided for box brushes. While the player is moving vertically, the only times they can collide with top or bottom sides is when they are either entering or leaving the brush along the vertical axis. However a small oversight made the that is chosen based on the player's direction and rather than the actual crossed

side, so even if they are crossing the bottom side, it will still be the topside that is checked. So while it is impossible to land on a topside while touching both brushes, it is still possible to briefly collide with the topside of the top brush while going past its bottom by abusing this small technicality. This is the idea behind a texturebug, which commonly happens whenever there is a change in texture on a wall, or rather, a change in brush. Because the top brush is collided with first anyway, it is not even necessary to include the bottom brush in the picture. That makes the bug totally free of the concept of a texture or brush change as the name of the bug otherwise suggests.

Time to figure out how a texturebug can actually happen in practice. When the player goes past the bottom of a box brush while also pressing up against one of its wall sides, the game will check Inequality (7). As the player is around the bottom of the brush, it means both s_z and f'_z will be negative. On the other hand, $s_\perp$ will be positive, and if $s_\perp \geq \varepsilon$, then the situation becomes $f'_z < 0 \leq f'_\perp$. The wall would always be chosen in this scenario, so for the bug to have a chance of working, s_z is required to be less than ε such that both $f'_\perp$ and f'_z are negative. The first step is therefore to setup for the texturebug by wiggling. The closer $s_\perp$ is to zero, the better. Both fractions will now be negative and the task is to make f'_z overtake $f'_\perp$ numerically. This can be achieved by manipulating $|\Delta_z|$ and $|\Delta_\perp|$. If the player is falling very fast, then the large value of $|\Delta_z|$ would scale the magnitude of f'_z down, but that actually ends up increasing its value because it is negative. A similar thing can be done for $f'_\perp$, where a small $|\Delta_\perp|$ decreases its value. To control $|\Delta_\perp|$, the wall-hug technique, that was explained at the start of Section 9, can be used to grant the player arbitrarily low speeds into the wall. Because it takes time for wall-hugging to lower $|\Delta_\perp|$, it means the player has to time pressing to the side such that they have a low enough speed when they go past the bottom of the brush. It was also mentioned that if a wall-hug is done for too long, then $|\Delta_\perp|$ will end up being zero and no collision will be able to take place. Texturebugs are therefore very strict with their timing.

The geometry of the brush itself also plays a role in how easy it is to texturebug on it. Because the goal is to maximize f'_z , having a less negative s_z is always better. A box bug with a lower height will therefore be easier to texturebug on since its bottom is closer to its top, effectively increasing the value of s_z . On the other hand, a very tall brush can end up becoming essentially impossible to texturebug on because s_z will be so negative that it would require unrealistic values for $|\Delta_z|$ and $|\Delta_\perp|$. By assuming $s_\perp = 0$, Inequality (7) can be changed into something neater for texturebugs:

$$\frac{H_{\text{brush}} + H_{\text{player}}}{\varepsilon} < \frac{|\Delta_z|}{|\Delta_\perp|} = \frac{|v_z|}{|v_\perp|}, \quad (10)$$

where H_{brush} and H_{player} are the height of the brush and the player's bounding box respectively. The condition is technically an approximation, but the terms that were done away with contributed almost nothing and the end result is a lot nicer because of it.

There seems to be somewhat of a discrepancy between how a texturebug is normally done by players and what is being described here. Players normally execute a texturebug by wall-hugging in one direction during the fall whereafter they change the direction when they get near the bottom of the brush. What they are actually doing is combining the setup wiggle with the texturebug wall-hug. The first long wall-hug they do gets them a bit closer to the wall, but the change in direction for the texturebug gets them even closer while also lowering the speed into the wall to make the texturebug work. So while a proper wiggle setup is best to ensure $s_\perp$ gets as close to zero as possible, the standard way of texturebugging ends up working well enough for most cases.